

Exercice 2 : Git et Docker

Rendu de la méthodologie Git, des fichiers Docker et de l'orchestration Docker Compose pour l'application DevOpsGPT.

ETUDIANT

Maëlis Nohé Estrada

MATIERE

DevOps

PROJET

DevOpsGPT

FORMATION

MSc Data Engineering & AI

1. Méthodologie et Git

L'équipe utilise Scrum. La fonctionnalité demandée est le système d'abonnement Premium.

User Story

En tant qu'utilisateur de DevOpsGPT, **je veux** pouvoir souscrire à un abonnement Premium **afin de** bénéficier de fonctionnalités avancées, de limites d'utilisation plus élevées et d'un accès prioritaire au service.

Créer la feature et faire un commit

```
git checkout main
git pull origin main
git checkout -b feature-premium-subscription
git add .
git commit -m "Add premium subscription feature"
git push -u origin feature-premium-subscription
```

Créer et pousser le tag v1.0.0

```
git checkout main
git pull origin main
git tag v1.0.0
git push origin v1.0.0
```

2. Dockerisation

Le projet est séparé en deux applications : un backend Node.js/Express et un frontend React/Vite. Chaque partie possède son propre Dockerfile.

backend/Dockerfile

```
FROM node:22-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

EXPOSE 3000

CMD ["node", "index.js"]
```

frontend/Dockerfile

```
FROM node:22-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

EXPOSE 5173

CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

3. Orchestration avec Docker Compose

Docker Compose permet de lancer ensemble le frontend, le backend et Redis. Les services communiquent entre eux par leurs noms de service.

chat-ui

Interface React/Vite utilisée par l'utilisateur.

Build : ./frontend

Port : 5173

Proxy : /api

chat-api

API Node.js/Express qui reçoit les messages et répond.

Build : ./backend

Port : 3000

Redis URL

cache

Service Redis utilisé pour stocker temporairement les messages.

Image : redis:7-alpine

Port : 6379

docker-compose.yml

```
services:
  chat-api:
    build: ./backend
    ports:
      - "3000:3000"
    environment:
      PORT: 3000
      REDIS_URL: redis://cache:6379
```

```
OPENAI_API_KEY: ${OPENAI_API_KEY:-}
```

```
depends_on:
```

```
- cache
```

```
chat-ui:
```

```
build: ./frontend
```

```
ports:
```

```
- "5173:5173"
```

```
depends_on:
```

```
- chat-api
```

```
cache:
```

```
image: redis:7-alpine
```

```
ports:
```

```
- "6379:6379"
```

Commande de lancement : `docker compose up --build` . Une fois les conteneurs démarrés, l'application est accessible sur `http://localhost:5173` .

Résumé des fichiers créés

FICHIER	RÔLE
backend/Dockerfile	Construit et lance l'API backend sur le port 3000.
frontend/Dockerfile	Construit et lance l'interface React/Vite sur le port 5173.
docker-compose.yml	Lance ensemble le backend, le frontend et Redis.